Parallelization

Using multiple cores and clusters

Why parallel computing exists

- Large scientific problems require enormous computation
- Examples:
  - Weather prediction
  - Real-time system simulations
  - Genome analysis
  - Protein folding

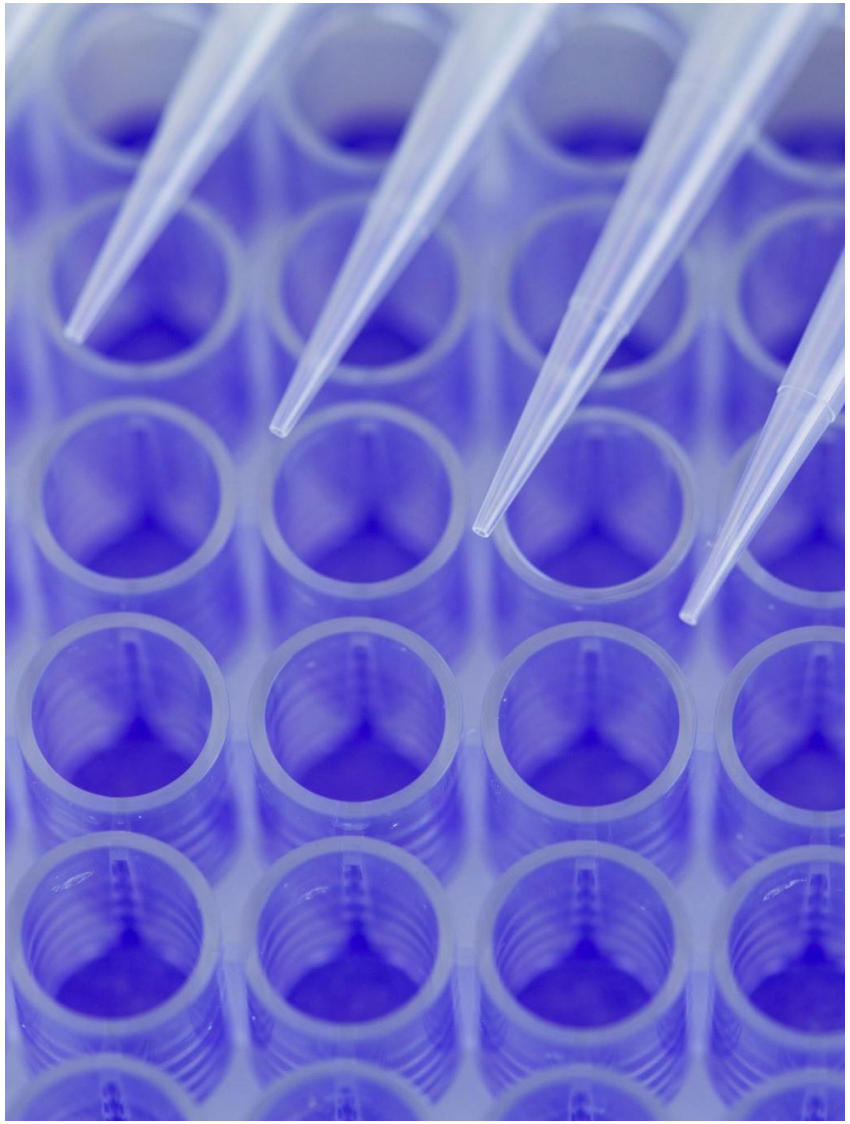
→ led to development of parallel hardware and high performance computing

Serial vs. Parallel

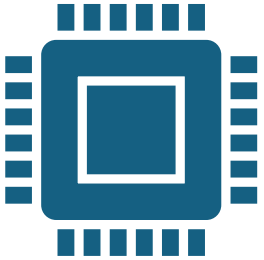
Task: pipette 1000 samples (10 sec per sample)

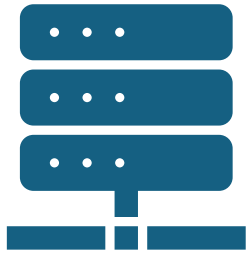
- Serial:
  - One person pipette every sample one by one: 2.8 hours
- Parallel:
  - 10 people, each takes 100 samples: 17 min

-> Same work finished much faster



Serial vs. Parallel Computing





Serial computing:

Tasks executed one after another

On a single CPU core

Simple but slow for large computations

Core: Task1 -> Task2 -> Task3

Parallel computing:

Tasks executed simultaneously

On multiple cores

Core1 -> Task1
Core2 -> Task2
Core3 -> Task3

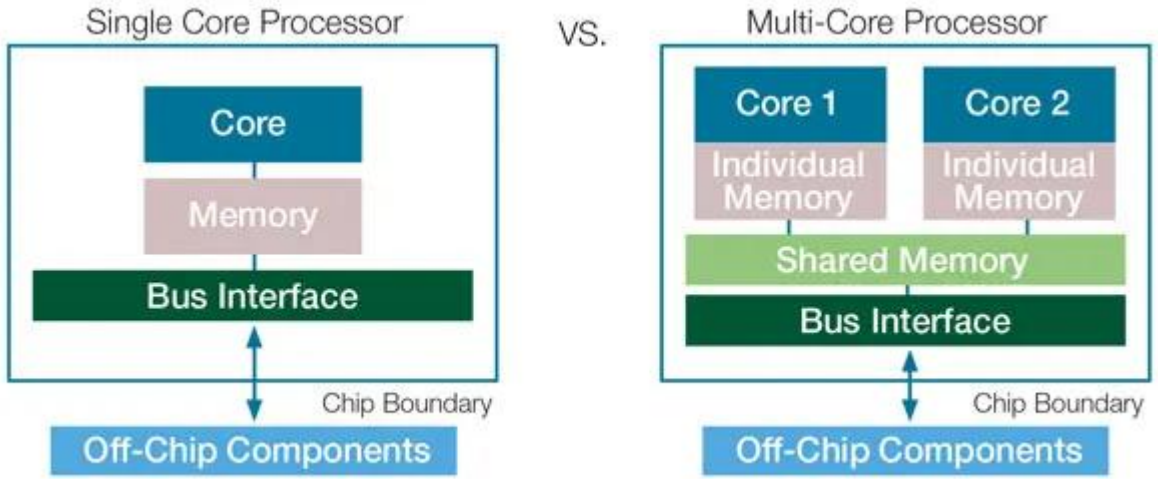
Terms

- Process:
  - Running instance of a program
  - Example: running R
- Thread:
  - Subtask inside a process
  - Multiple threads can run in parallel
- Core:
  - Physical computing unit of the CPU
  - Executes threads

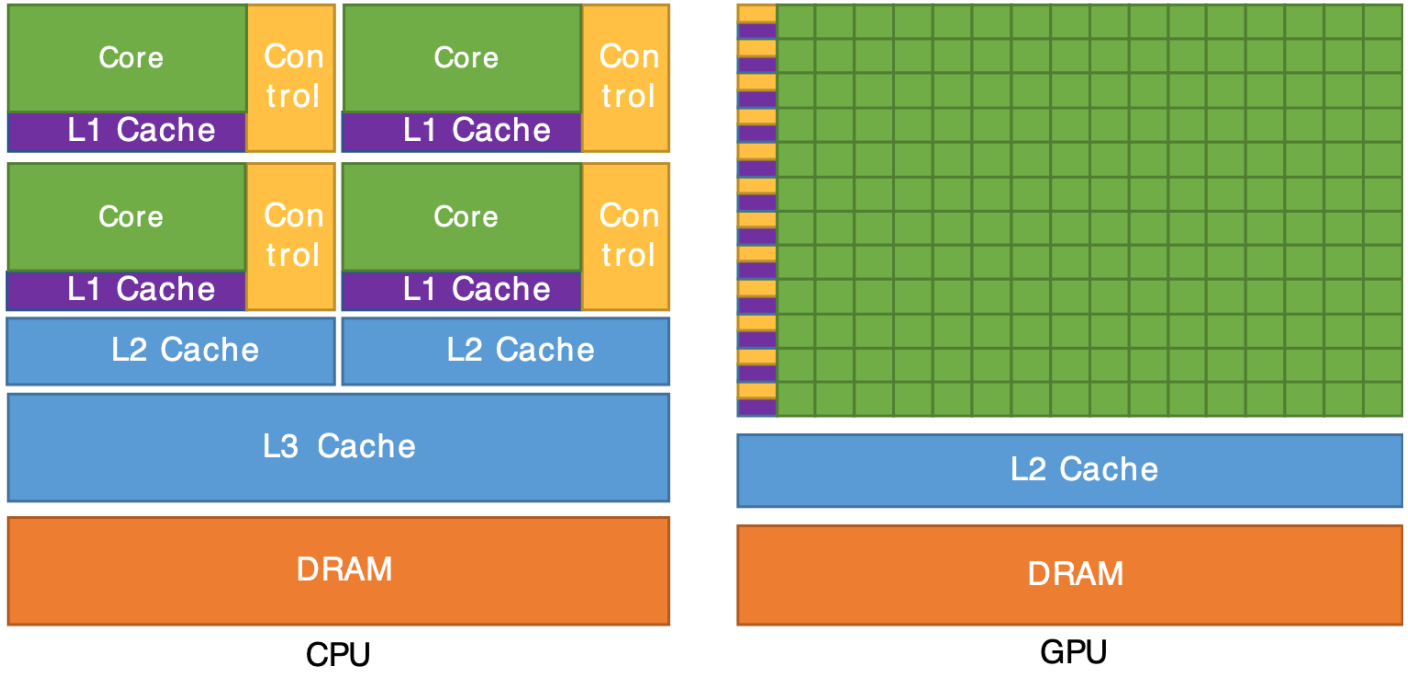
Process
↓
Threads
↓
Cores

Architecture

- Modern computers have multiple CPU cores
  - Example: laptop: 4-16 cores
  - Number of cores on your hardware: parallel::detectCores() in R
- Each core can run tasks simultaneously
- But software must be written to use multiple cores (parallelize code)



CPUs vs. GPUs



CPU (Central Processing Unit)

- Few powerful cores
- Optimized for complex logic and sequential tasks
- Large cache and sophisticated control units
- Example tasks: running programs, statistics, data processing

GPU (Graphics Processing Unit)

- Thousands of smaller cores (for simple independent tasks)
- Optimized for repeating operations
- Very efficient for matrix operations
- Example tasks: deep learning, image processing, simulations

Types of parallelism

- Embarrassingly Parallel:
  - Task can be divided into independent parallel tasks
  - Workload can be distributed across multiple processors without complex coordination need
- Task Parallelism:
  - Decompose task into subtasks
  - Run different tasks simultaneously
- Data Parallelism:
  - Same operation on different data

Overhead

- Parallel programs are not always faster

Reasons:

- Communication:
  - Sending data between cores takes time
- Synchronization:
  - Cores sometimes have to "wait" for each other at a finish line before moving to the next step (called a barrier)
- Context Switching:
  - Too many threads slow the system

Amdahl‘s Law

Optimally: speedup from parallelization would be linear: doubling number of processing elements should have halve the runtime

Max speedup limited by serial fraction of code (part that can not be parallelized, e.g. loading file / final data merging)

$$S = \frac{1}{(1-p)+\frac{p}{n}}$$

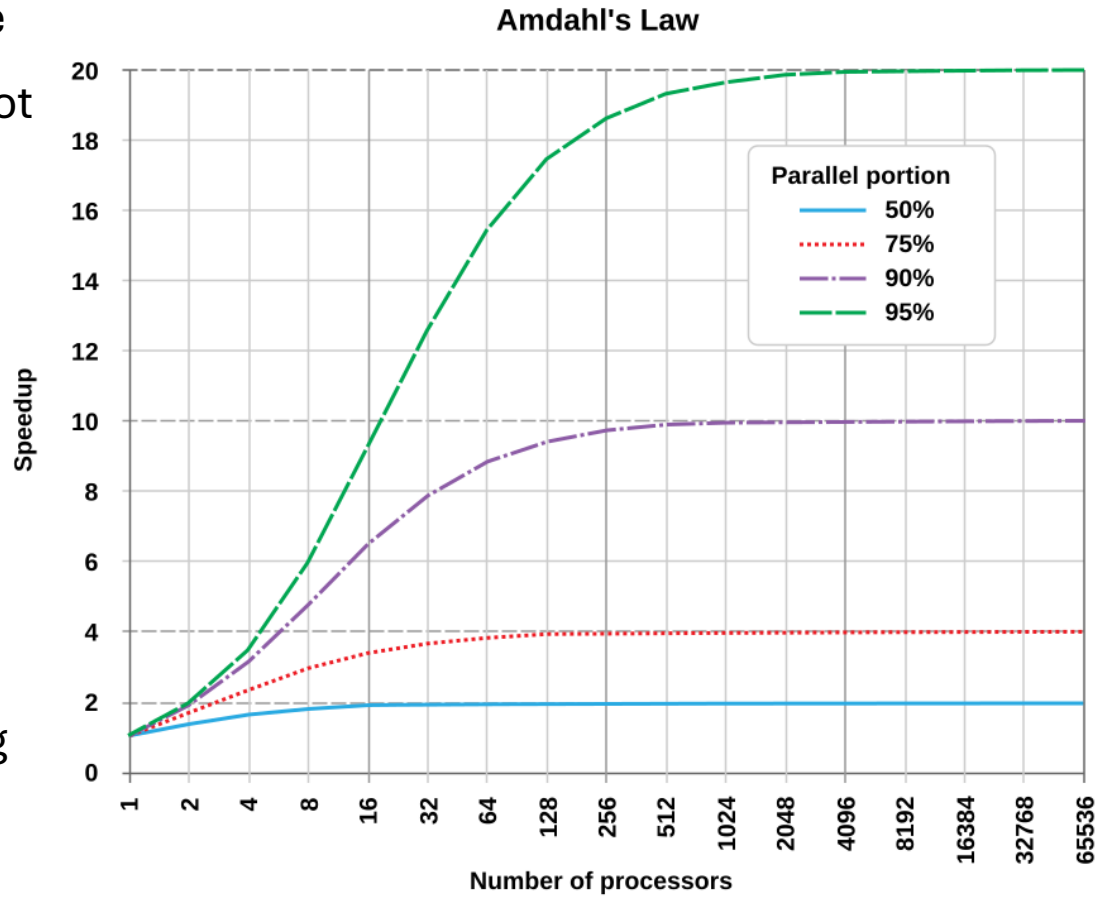
S: Overall speedup

p: Portion of code that can be parallelized

n: Number of cores

Example:
Script spends 20% on loading genome (serial), 80% on aligning reads (parallel). What is the overall speedup if $n = 10$?

What if $n \rightarrow \infty$?



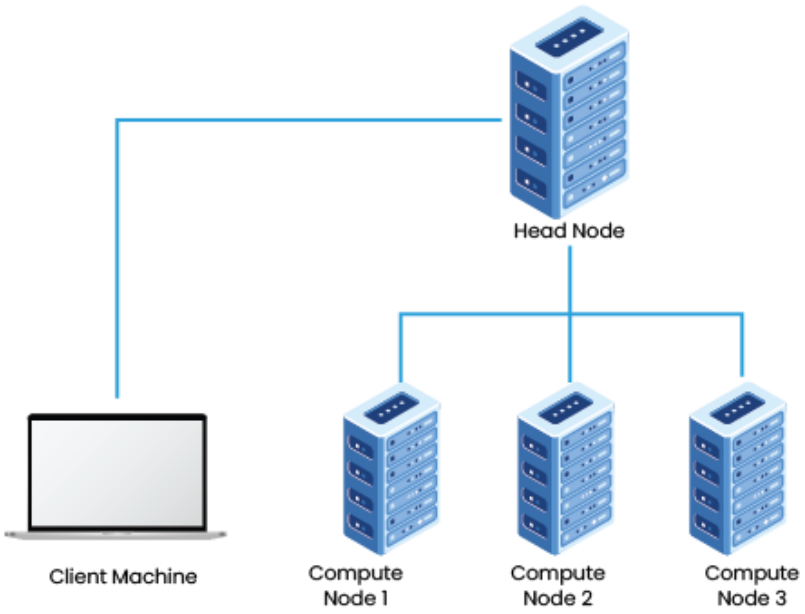
➔Even with infinite number of cores, code will never be more than 5 times faster

Gustafson‘s Law

- suggests that as we get more cores, we don't just want to run the same task faster; we want to run a bigger task in the same amount of time
- Amdahl: "How can I finish this one sequence alignment faster?“
- Gustafson: "In the same hour, how many more sequences can I align?“
- Instead of making one task faster: Use more cores to solve larger problems

Local vs. Cluster

- Parallelization on Your Computer
  - Uses multiple CPU cores (sometimes GPU possible) on one computer
  - All cores share the same memory
  - Example: 8 cores on laptop
- Cluster Computing:
  - Many computers connected together
  - Each node basically a full computer (CPU + RAM) with its own memory
  - Communication between nodes slower than between cores
  - Some clusters also contain GPUs (e.g.: Node1: CPU, RAM, GPU)
  - Cluster Jobs are managed by a scheduler (e.g. SLURM, PBS,...)
  - Example: 1000+ CPU cores; TBs of RAM



Parallel Programming in R

Parallel Programming approaches:

- Multi-threading:
  - Execution of multiple threads of a single process simultaneously
- Multi-processing:
  - Execution of multiple processes simultaneously
- Distributed computing:
  - Distribution of large computational task across multiple computers connected to a network

Parallel Programming in R

- Packages: parallel, future, foreach doMC
- Find out how many cores your computer has: detectCores()
- Claude:
  - parallel:
    - For simple, independent loops on Linux/Mac
      Key functions: mclapply(), parLapply()
  - foreach:
    - For loop-style-parallelism
    - Key functions: foreach(...) %dopar% { ... }
  - future:
    - For modern, flexible parallism (works locally and on cluster)
    - Write the code once and just change plan(multisession) to plan(cluster)
  - BiocParallel:
    - For Bioconducter workflows (DESeq2,...)

When to use parallelization? - Dependencies

- No program can run more quickly than the longest chain of dependent calculations (= critical path)
- Berstein‘s condition:
  P_i and P_j are independent (and can be executed in parallel) if:

$$I_j \cap O_i = \emptyset$$
$$I_i \cap O_j = \emptyset$$
$$O_i \cap O_j = \emptyset$$

  with I_i all input variables of P_i, O_i all output variables of P_i,
  I_j all input variables of P_j, O_j all output variables of P_j

Discussion

- When does it makes sense to use parallelization?
- What tasks could be parallelized (in our lab)?
- When is a cluster necessary?
- When is local parallelization enough?
- What is your experience with parallelization? What do you need to keep in mind when parallelizing?